SkillBarter — Beast Mode Testing Prompt Suite

Production-Grade Testing Handbook for the Peer-to-Peer Skill Exchange Platform

2025–2026

Table of Contents

1. Section 1 — How to Use This Guide
    1. Model Selection Strategy (Claude Opus 4.5 / Sonnet 4.5 / Gemini 2.5 Pro)
    2. Beast Mode Tips & System Instruction Template
2. Section 2 — Phase-Wise Testing Prompts
    3. Phase 1 — Project Setup + Auth + DB Schema
    4. Phase 2 — Token Economy (LedgerService + Wallet Dashboard)
    5. Phase 3 — Marketplace + Search + Matching + Booking
    6. Phase 4 — Live Classroom (WebRTC + Monaco + Excalidraw)
    7. Phase 5 — Bounty Board (Async Video Q&A)
    8. Phase 6 — Notifications + WebSocket + Online Presence
    9. Phase 7 — Admin Panel + Dispute Resolution
    10. Phase 8 — DevOps: Docker + Redis + CI/CD + Production Hardening
3. Section 3 — The Mega End-to-End Prompt
    11. Part 1: The Golden Path Simulation (15 Steps)
    12. Part 2: Security Audit
    13. Part 3: Token Conservation Audit
    14. Part 4: Production Readiness Check
4. Section 4 — UX Quality Gates Checklist
    15. Loading States, Error Boundaries, Toast Notifications
    16. Mobile Responsiveness, Empty States, Disabled States

17. Optimistic Updates, Accessibility, Visual Quality
5. Section 5 — Token Conservation Audit SQL Queries
18. 8 Diagnostic SQL Queries with Explanations

Section 1 — How to Use This Guide

This handbook is the authoritative testing reference for SkillBarter — a peer-to-peer skill exchange platform powered by a SkillCoin token economy. It is structured as a phase-by-phase prompt suite designed to be pasted directly into an AI coding assistant (Claude Opus 4.5, Claude Sonnet 4.5, or Gemini 2.5 Pro). Each prompt is self-contained, precise, and written to surface bugs, security vulnerabilities, and financial logic errors on the first pass.

> WORKFLOW: Execute phases in order. Each phase depends on the previous. Do not skip Phase 2 (LedgerService) — it is the financial foundation. A bug in LedgerService means real token loss.

1.1 Model Selection Strategy

TABLE 1 — AI MODEL SELECTION BY PHASE

Model	Use For Phases	Reasoning	Key Capability
Claude Opus 4.5	Phases 2, 4, 7	Deepest financial logic, complex multi-step flows, dispute math	Enable extended thinking. Best for LedgerService unit tests, escrow state machines, dispute resolution math, admin ban transactions.
Claude Sonnet 4.5	Phases 1, 3, 5, 6	Fast iterative testing, API endpoint audits, DTO validation, frontend component checks	Great for rapid fix-and-verify loops. Best for auth flows, marketplace search, bounty board, WebSocket/notifications.
Gemini 2.5 Pro	Phase 8 + Mega E2E	Best at ingesting large multi-file contexts simultaneously	Use its 1M token context window to load the entire codebase. Best for Docker builds, Nginx config, GitHub Actions pipelines, env var validation.

1.2 Beast Mode Tips

6. Load full file contents into context — paste the entire file, not summaries. Summaries cause the model to hallucinate missing details.
7. Chain prompts — use the previous AI output (including its bug list) as the next prompt's context. This creates a cumulative audit trail.

8. Instruct the model to fix bugs inline — not just report them. The system instruction template below enforces this.
9. For financial logic (Phase 2, 4, 7): always enable extended thinking in Claude Opus 4.5. Financial bugs require step-by-step reasoning before conclusions.
10. For Phase 8 (DevOps): load ALL config files simultaneously into Gemini 2.5 Pro. Its 1M token window handles the full codebase in one pass.

1.3 Opening System Instruction Template

Paste the following system instruction at the top of every testing session. Customize the phase-specific focus area as indicated:

> SYSTEM INSTRUCTION: "You are a senior full-stack engineer auditing the SkillBarter platform for production readiness. For every bug you find: (1) describe it precisely, (2) assign severity — CRITICAL (security breach / data loss) / HIGH (broken feature) / MEDIUM (edge case) / LOW (code quality), (3) provide the corrected code inline. Do not stop at reporting — fix everything. Reason step-by-step through each flow before concluding. Enable extended thinking for all financial logic."

Section 2 — Phase-Wise Testing Prompts

Phase 1 — Project Setup + Auth + DB Schema

TABLE 2 — PHASE 1 MODEL CONFIGURATION

Attribute	Value
Recommended Model	Claude Sonnet 4.5
Reasoning	Fast iterative testing of auth flows, DTO validation, and schema constraints.
Extended Thinking	Not required — auth flows are well-defined state machines.

Files to Load

- apps/backend/src/auth/auth.service.ts
- apps/backend/src/auth/auth.controller.ts
- apps/backend/src/auth/auth.module.ts
- apps/backend/src/auth/strategies/jwt.strategy.ts
- apps/backend/src/auth/strategies/github.strategy.ts
- apps/backend/src/auth/strategies/linkedin.strategy.ts
- apps/backend/src/auth/dto/register.dto.ts
- apps/backend/src/auth/dto/login.dto.ts
- apps/backend/prisma/schema.prisma
- apps/frontend/src/store/authStore.ts
- apps/frontend/src/lib/axios.ts
- apps/frontend/src/components/ProtectedRoute.tsx
- apps/frontend/src/pages/auth/RegisterPage.tsx
- apps/frontend/src/pages/auth/LoginPage.tsx

Beast Mode Instruction

> "You are a senior security engineer auditing the SkillBarter authentication system for production readiness. For every bug, vulnerability, or deviation from spec you find: 1. Describe the issue precisely. 2. Assign severity: CRITICAL / HIGH / MEDIUM / LOW. 3. Provide the corrected code inline — do not just describe the fix, write it. Reason step-by-step through each flow before concluding."

Testing Prompt — Phase 1 Test Scenarios

1. Registration Flow

- Simulate: POST /auth/register with valid payload { email, password, name }
- Verify: bcrypt rounds = 12 (not 10, not 14 — exactly 12)
- Verify: 50-coin signup bonus creates a Transaction record with type=EARN, amount=50, userId=newUser.id
- Verify: token_balance starts at 50 (not 0, not 100)
- Verify: passwordHash is NEVER returned in the response body
- Verify: ValidationPipe has whitelist: true AND forbidNonWhitelisted: true
- Edge case: Register with extra field { email, password, name, isAdmin: true } — must be stripped/rejected
- Edge case: Register with same email twice — must return 409 Conflict, not 500

2. JWT Token Handling

- Verify: Access token is returned in response BODY (not Set-Cookie)
- Verify: Refresh token is set as httpOnly cookie (not in response body)
- Verify: Access token expiry = 15 minutes (900 seconds)
- Verify: Refresh token expiry = 7 days (604800 seconds)
- Verify: Frontend authStore stores ONLY the user object (id, email, name, role) — NOT the token string
- Verify: Token is stored in memory (Zustand store) — NOT in localStorage or sessionStorage
- BUG CHECK: Search entire frontend codebase for localStorage.setItem — must return 0 results for token storage
- Verify: Axios interceptor catches 401 responses, calls /auth/refresh, retries original request once

3. OAuth2 Idempotency

- Simulate: User connects GitHub OAuth for the first time → verify +50 coins credited, Transaction record created
- Simulate: Same user connects GitHub OAuth again → verify 0 additional coins (idempotent)
- Simulate: User connects LinkedIn OAuth → verify +50 coins credited
- Simulate: Same user connects LinkedIn OAuth again → verify 0 additional coins
- Verify: OAuth strategy checks if provider already linked before crediting bonus
- Verify: Total max OAuth bonus = 100 coins (50 GitHub + 50 LinkedIn), never more

4. ProtectedRoute Behavior

- Simulate: Unauthenticated user navigates to /dashboard → verify redirect to /login
- Simulate: Authenticated user navigates to /login → verify redirect to /dashboard (no re-login loop)
- Verify: ProtectedRoute checks Zustand authStore, not localStorage

5. Prisma Schema Constraints

- Verify: token_balance has CHECK constraint >= 0 (cannot go negative at DB level)
- Verify: User.passwordHash field is NOT selected by default in any Prisma query that returns user to frontend
- Verify: All FK relationships have correct onDelete behavior:
  - Session → User (mentor/student): onDelete Restrict
  - Transaction → User: onDelete Cascade
  - Skill → User: onDelete Cascade
- Verify: GIN index exists on skills.search_vector
- Verify: Composite index exists on sessions(mentor_id, start_time)
- Verify: All enum types match implementation (SessionStatus, TransactionType, EscrowStatus, BountyStatus)

6. Security Checks

- Attempt: GET /users/me without Authorization header → must return 401
- Attempt: GET /users/me with malformed JWT → must return 401 (not 500)
- Attempt: GET /users/me with expired JWT → must return 401 with message 'Token expired'
- Verify: No route accidentally exposes passwordHash in response

Critical Rules Checklist — Phase 1

11. bcrypt rounds = exactly 12
12. 50-coin signup bonus creates Transaction record (type=EARN)
13. JWT access token in response body (memory only, never localStorage)
14. Refresh token in httpOnly cookie only
15. OAuth bonus credited exactly once per provider (idempotent)
16. passwordHash never in any API response
17. ProtectedRoute redirects unauthenticated users to /login
18. Zustand AuthStore stores only user object (not token)
19. Axios interceptor handles 401 with refresh + retry
20. ValidationPipe has whitelist: true AND forbidNonWhitelisted: true

21. token_balance CHECK constraint >= 0 in Prisma schema
22. FK onDelete behaviors correct for all relationships
23. GIN index on search_vector exists

Phase 2 — Token Economy (LedgerService + Wallet Dashboard)

TABLE 3 — PHASE 2 MODEL CONFIGURATION

Attribute	Value
Recommended Model	Claude Opus 4.5 with Extended Thinking
Reasoning	Financial logic requires deep reasoning. LedgerService is the most critical file — bugs here cause real token loss.
Extended Thinking	REQUIRED — enable before loading files.

> CRITICAL: This is the most important phase in the entire test suite. A single non-atomic operation in LedgerService can cause token duplication or destruction. Every financial operation MUST be wrapped in prisma.$transaction().

Files to Load

- apps/backend/src/ledger/ledger.service.ts
- apps/backend/src/ledger/ledger.service.spec.ts
- apps/backend/src/wallet/wallet.controller.ts
- apps/backend/src/wallet/wallet.service.ts
- apps/backend/prisma/schema.prisma (Transaction + Escrow models)
- apps/frontend/src/pages/wallet/WalletPage.tsx
- apps/frontend/src/components/wallet/BalanceCard.tsx
- apps/frontend/src/components/wallet/TransactionHistoryTable.tsx
- apps/frontend/src/hooks/useWallet.ts

Beast Mode Instruction

> "You are a senior fintech engineer auditing the SkillBarter token economy for production readiness. This is the most critical part of the codebase — bugs here cause token loss or creation out of thin air. Enable extended thinking. Before answering any question about financial logic: 1. Trace the full transaction flow step by step. 2. Verify atomicity (is this wrapped in prisma.$transaction()?). 3. Verify math (is tax Math.floor, not Math.round?). 4. Verify state machine (are escrow transitions one-way?). 5. Then provide your conclusion. For every bug: assign severity and provide corrected code inline."

Testing Prompt — Phase 2 Test Scenarios

1. LedgerService Unit Tests (All 7 Must Pass)

TABLE 4 — LEDGERSERVICE UNIT TEST MATRIX

Test #	Method	Assertion
1	credit()	Balance increases + Transaction record created with type=EARN
2	debit() — success	Balance decreases + Transaction record created with type=SPEND
3	debit() — insufficient funds	Throws BadRequestException when balance < amount
4	lockEscrow()	Escrow created with status=LOCKED, user balance decremented atomically
5	releaseEscrow()	Escrow → RELEASED, mentor credited, 5% tax to admin, teachingStreak++
6	refundEscrow()	Escrow → REFUNDED, student balance restored in full
7	applyNoShowPenalty()	Penalty = Math.min(10, mentorBalance) — never goes below 0

2. Atomicity Verification

- Verify: lockEscrow uses prisma.$transaction() — NOT sequential await calls
- Verify: releaseEscrow uses prisma.$transaction() — NOT sequential await calls
- Verify: refundEscrow uses prisma.$transaction() — NOT sequential await calls
- Verify: splitEscrow uses prisma.$transaction() — NOT sequential await calls
- Verify: creditUser signup bonus uses prisma.$transaction() — NOT sequential await calls
- RED FLAG: Search for any pattern like: await prisma.user.update(...balance); await prisma.transaction.create(...) — this is NOT atomic and must be fixed immediately

3. Tax Math Verification

TABLE 5 — TAX MATH TEST CASES (ALWAYS MATH.FLOOR, NEVER MATH.ROUND)

Input Amount	Expected Tax (Math.floor)	Mentor Receives	Verification
100 coins	Math.floor(5.00) = 5	95 coins	PASS
101 coins	Math.floor(5.05) = 5	96 coins	PASS

99 coins	Math.floor(4.95) = 4	95 coins	PASS
1 coin	Math.floor(0.05) = 0	1 coin	PASS — no tax on tiny amounts
19 coins	Math.floor(0.95) = 0	19 coins	PASS — floor prevents over-taxation

4. Escrow State Machine

TABLE 6 — ESCROW STATE MACHINE TRANSITION MATRIX

Transition	Valid?	resolvedAt Set?	Notes
LOCKED → RELEASED	YES	YES	Normal session completion
LOCKED → DISPUTED	YES	NO	Dispute raised — admin must resolve
LOCKED → REFUNDED	YES	YES	Cancellation or no-show
RELEASED → LOCKED	NO — must throw	N/A	One-way state machine violation
DISPUTED → LOCKED	NO — must throw	N/A	One-way state machine violation
REFUNDED → LOCKED	NO — must throw	N/A	One-way state machine violation
DISPUTED → RELEASED	YES (admin only)	YES	Admin resolves FAVOR_MENTOR or SPLIT
DISPUTED → REFUNDED	YES (admin only)	YES	Admin resolves FAVOR_STUDENT

5. Admin Wallet & Teaching Streak

- Verify: Admin wallet is a real User record in the database (not a hardcoded ID like 0 or -1)
- Verify: Admin wallet userId is loaded from environment variable or config (not hardcoded)
- Verify: Tax transactions create a Transaction record: { type: TAX, fromUserId: mentor, toUserId: adminWallet, amount: tax }
- Verify: releaseEscrow increments mentor.teachingStreak in the SAME prisma.$transaction()
- Verify: teachingStreak is NOT incremented in refundEscrow
- Verify: teachingStreak is NOT incremented in splitEscrow (dispute resolution)

Critical Rules Checklist — Phase 2

24. All 7 Jest unit tests pass

25. prisma.$transaction() used for EVERY financial operation (never sequential awaits)
26. tax = Math.floor(amount × 0.05) — never Math.round or Math.ceil
27. tokenBalance never goes below 0 (application-level check before DB constraint fires)
28. Escrow state transitions are one-way (LOCKED → RELEASED/DISPUTED/REFUNDED only)
29. Admin wallet is a real DB user (not a magic number or hardcoded ID)
30. releaseEscrow increments teachingStreak in same transaction
31. freezeEscrow does NOT set resolvedAt
32. disputeEscrow does NOT set resolvedAt
33. Transaction record created for EVERY balance change
34. BalanceCard uses animated count-up (not instant number change)
35. TransactionHistoryTable badge colors correct (EARN=green, SPEND=red, TAX=gray)
36. TanStack Query staleTime = 30_000 on all wallet queries

Phase 3 — Marketplace + Search + Matching + Booking

TABLE 7 — PHASE 3 MODEL CONFIGURATION

Attribute	Value
Recommended Model	Claude Sonnet 4.5
Reasoning	Fast iteration needed for API endpoint audits, DTO validation, and frontend component checks.
Extended Thinking	Not required — but enable for booking atomicity analysis.

Files to Load

- apps/backend/src/skills/skills.service.ts
- apps/backend/src/skills/skills.controller.ts
- apps/backend/src/skills/dto/create-skill.dto.ts
- apps/backend/src/skills/dto/search-skills.dto.ts
- apps/backend/src/sessions/sessions.service.ts
- apps/backend/src/sessions/sessions.controller.ts
- apps/backend/src/sessions/dto/create-session.dto.ts
- apps/backend/prisma/schema.prisma (Skill + Session models)
- apps/frontend/src/pages/marketplace/MarketplacePage.tsx
- apps/frontend/src/components/marketplace/SkillCard.tsx
- apps/frontend/src/components/marketplace/BookingModal.tsx

- apps/frontend/src/hooks/useMarketplace.ts
- apps/frontend/src/hooks/useBooking.ts

Beast Mode Instruction

> "You are a senior backend engineer auditing the SkillBarter marketplace and booking system. Focus on: atomicity of booking transactions, search correctness, and the 5 booking validation rules. For every bug: assign severity and provide corrected code inline. Pay special attention to race conditions in concurrent booking scenarios."

Testing Prompt — Phase 3 Test Scenarios

1. Full-Text Search

- Verify: GIN index exists on skills.search_vector in Prisma schema
- Verify: search_vector is updated via Postgres trigger on skill insert/update
- Test: Search 'javascript' → returns skills with 'JavaScript' in title/description/tags
- Test: Search 'react typescript' → returns skills matching both terms
- Test: Search with empty string → returns all skills (no error)
- Test: Search with SQL injection attempt: "'; DROP TABLE skills; --" → sanitized, no error
- Verify: Search uses to_tsquery or plainto_tsquery (not LIKE %%) for performance
- Verify: Results ordered by ts_rank DESC (most relevant first)

2. The 5 Booking Validation Rules (ALL Must Be Enforced)

TABLE 8 — THE 5 MANDATORY BOOKING VALIDATION RULES

Rule #	Validation	Expected Error on Violation
1	Student cannot book their own skill (self-booking blocked)	400 Bad Request: 'Cannot book your own skill'
2	Student balance must be >= skill price	400 Bad Request: 'Insufficient balance'
3	No time overlap with existing sessions for mentor	409 Conflict: 'Mentor unavailable at this time'
4	No time overlap with existing sessions for student	409 Conflict: 'You have a conflicting session'
5	Session start_time must be in the future	400 Bad Request: 'Cannot book past time slots'

3. Booking Atomicity — THE MOST CRITICAL TEST

- Verify: Booking creation wraps BOTH operations in single prisma.$transaction(): (1) LedgerService.lockEscrow() AND (2) prisma.session.create()
- RED FLAG: If these are sequential awaits (not in transaction), a crash between them causes permanent token loss
- Verify: Overlap check runs BEFORE escrow lock (not after)
- Simulate race condition: Two students book same mentor slot simultaneously → only one succeeds, the other receives 409

4. Skill Soft Delete

- Verify: DELETE /skills/:id performs soft delete (sets isActive=false, does NOT remove row)
- Verify: Soft-deleted skills do NOT appear in marketplace search results
- Verify: Hard delete is blocked (no prisma.skill.delete() calls anywhere in codebase)
- Verify: Existing sessions for soft-deleted skills are NOT cancelled (historical data preserved)

Critical Rules Checklist — Phase 3

37. GIN index on search_vector exists
38. FTS search returns correct results (not LIKE query)
39. Booking atomicity: escrow lock + session create in single prisma.$transaction()
40. Overlap check runs BEFORE escrow lock
41. All 5 booking validations enforced with correct error messages
42. Soft delete for skills (hard delete blocked)
43. passwordHash never in skill/user responses
44. URL params reflect filter state (shareable URLs)
45. BookingModal disabled when balance < price
46. TanStack Query staleTime = 30_000 on marketplace queries
47. Search debounced 300ms
48. Mentor notified via WebSocket on successful booking

Phase 4 — Live Classroom (WebRTC + Monaco + Excalidraw)

TABLE 9 — PHASE 4 MODEL CONFIGURATION

Attribute	Value
Recommended Model	Claude Opus 4.5 with Extended Thinking
Reasoning	Session state machine + escrow release + dispute flow from classroom involves complex multi-step financial

	logic.
Extended Thinking	REQUIRED — trace the full completion flow step by step.

Files to Load

- apps/backend/src/sessions/sessions.service.ts (joinRoom, completeSession, raiseDispute methods)
- apps/backend/src/livekit/livekit.service.ts
- apps/backend/src/livekit/livekit.controller.ts
- apps/backend/src/disputes/disputes.service.ts
- apps/frontend/src/pages/classroom/ClassroomPage.tsx
- apps/frontend/src/components/classroom/VideoGrid.tsx
- apps/frontend/src/components/classroom/CodeEditor.tsx (Monaco)
- apps/frontend/src/components/classroom/Whiteboard.tsx (Excalidraw)
- apps/frontend/src/components/classroom/CountdownTimer.tsx
- apps/frontend/src/components/classroom/RatingModal.tsx
- apps/frontend/src/hooks/useLiveKit.ts

Beast Mode Instruction

> "You are a senior WebRTC engineer auditing the SkillBarter live classroom system. Enable extended thinking. For the session completion flow, trace every step: 1. Both users click 'Complete'. 2. Session status transitions. 3. Escrow release with tax. 4. teachingStreak increment. 5. Rating modal appearance. All 5 steps must happen atomically or with correct error handling. For every bug: assign severity and provide corrected code inline."

Testing Prompt — Phase 4 Test Scenarios

1. LiveKit Room Token

- Verify: GET /livekit/token?sessionId=X returns a valid LiveKit JWT
- Verify: Token includes correct roomName (session ID or derived name)
- Verify: Token includes correct participantName (user's display name)
- Verify: Token expires appropriately (not infinite TTL)
- Verify: Only session participants (mentor + student) can get tokens for that room
- Attempt: Get token for session you're not part of → must return 403 Forbidden

2. Both-Party Completion + Escrow Release (Atomic Flow)

- Simulate: Mentor clicks 'Complete' → verify mentor_completed = true, escrow NOT yet released
- Simulate: Student clicks 'Complete' → verify student_completed = true, THEN verify ALL of the following in single prisma.$transaction():
  - Session status → COMPLETED
  - LedgerService.releaseEscrow() called
  - 5% tax Math.floor'd to admin wallet
  - Mentor balance credited (amount - tax)
  - mentor.teachingStreak++ in SAME transaction
  - resolvedAt set on escrow
- Verify: If only one party marks complete, status stays ACTIVE (not COMPLETED)
- Verify: Session cannot go from COMPLETED back to ACTIVE

3. Dispute Flow from Classroom

- Simulate: User clicks 'Raise Dispute' during active session
- Verify: Dispute record created in DB
- Verify: Escrow status → DISPUTED (not LOCKED, not RELEASED)
- Verify: resolvedAt NOT set on escrow when dispute raised
- Verify: Admin receives notification (WebSocket or DB notification)
- Verify: Both participants see 'Dispute Raised' UI state
- Verify: Session cannot be 'completed' while in DISPUTED state

4. Join Room Button State

- Verify: 'Join Room' button is DISABLED more than 10 minutes before session start_time
- Verify: 'Join Room' button becomes ACTIVE exactly 10 minutes before start_time
- Verify: Button state updates in real-time (not requiring page refresh)
- Verify: Countdown timer shows time until session starts

Critical Rules Checklist — Phase 4

49. LiveKit room token generation correct (time-gated on backend, never client-side)
50. 'Join Room' button only active 10 min before start_time
51. Session status: PENDING → ACTIVE → COMPLETED (one-way transitions only)
52. Both-party completion required before escrow release
53. Countdown timer synced via LiveKit data channel (not independent JS timers)
54. Monaco Editor loads and syncs correctly between participants

55. Excalidraw loads and syncs correctly between participants
56. Dispute flow: raises dispute, freezes escrow (DISPUTED), notifies admin
57. Rating modal appears after completion (not during dispute)
58. Split layout responsive across desktop/tablet/mobile

Phase 5 — Bounty Board (Async Video Q&A)

TABLE 10 — PHASE 5 MODEL CONFIGURATION

Attribute	Value
Recommended Model	Claude Sonnet 4.5
Reasoning	Bounty flows are well-defined state machines. Fast iteration needed for upload flow, escrow logic, and expiry cron job.
Extended Thinking	Not required.

Files to Load

- apps/backend/src/bounties/bounties.service.ts
- apps/backend/src/bounties/bounties.controller.ts
- apps/backend/src/bounties/dto/create-bounty.dto.ts
- apps/backend/src/bounties/bounty-expiry.cron.ts
- apps/backend/src/storage/storage.service.ts (presigned URL generation)
- apps/frontend/src/pages/bounties/BountyBoardPage.tsx
- apps/frontend/src/components/bounties/CreateBountyModal.tsx
- apps/frontend/src/components/bounties/BountyCard.tsx
- apps/frontend/src/components/bounties/VideoAnswerUploader.tsx
- apps/frontend/src/components/bounties/BountyDetailModal.tsx

Beast Mode Instruction

> "You are a senior engineer auditing the SkillBarter bounty board system. Key focus areas: presigned URL upload flow (file must NEVER route through NestJS), escrow lifecycle for bounties, and the 7-day expiry cron job idempotency. For every bug: assign severity and provide corrected code inline."

Testing Prompt — Phase 5 Test Scenarios

1. Presigned URL Upload Flow

- Verify: GET /storage/presigned-url returns a presigned URL (S3 or Cloudinary)
- Verify: Video file is uploaded DIRECTLY from browser to S3/Cloudinary (not through NestJS)
- Verify: NestJS never receives the video file bytes (no multer/file buffer in controller)
- Verify: After upload, client sends only the file URL/key to NestJS (not the file)
- Verify: File size limit enforced at storage provider level (not NestJS)
- RED FLAG: Any controller with @UploadedFile() for video answers is WRONG — this routes the file through NestJS memory

2. Bounty Status Transitions

TABLE 11 — BOUNTY STATUS STATE MACHINE

From	To	Trigger	Tax Applied?
OPEN	ANSWERED	First answer submitted	No
ANSWERED	CLOSED	Poster accepts an answer	Yes — 5% Math.floor
OPEN	EXPIRED	7-day cron job runs	No — full refund to poster
ANSWERED	EXPIRED	7-day cron job runs	No — full refund to poster
CLOSED	Any	Any action	BLOCKED — cannot transition from CLOSED

3. 7-Day Expiry Cron Job

- Verify: Cron runs on schedule (EVERY_HOUR as per spec)
- Verify: Finds all bounties where status IN [OPEN, ANSWERED] AND expiresAt < now()
- Verify: For each expired bounty — ALL of the following in single prisma.$transaction():
  - Escrow refunded to poster in full (NO tax on expiry refund)
  - Bounty status → EXPIRED
  - Poster notified (WebSocket or DB notification)
- Verify: Cron is IDEMPOTENT — checks escrow.status === LOCKED before processing (skips already-processed bounties)
- Verify: Cron does NOT expire CLOSED bounties

Critical Rules Checklist — Phase 5

59. Bounty creation validates balance >= rewardCoins BEFORE creating bounty
60. Escrow locked atomically with bounty creation (single prisma.$transaction())
61. Presigned URL upload: file NEVER routes through NestJS
62. No @UploadedFile() decorator on video answer submission endpoint
63. Accept answer triggers LedgerService escrow release with 5% tax
64. Only ONE answer can be accepted per bounty (enforced on backend)
65. 7-day expiry cron refunds escrow in full (no tax on expiry)
66. Cron is idempotent (checks escrow.status === LOCKED before processing)
67. Expired bounty notifies poster
68. BountyStatus EXPIRED set correctly by cron

Phase 6 — Notifications + WebSocket + Online Presence

TABLE 12 — PHASE 6 MODEL CONFIGURATION

Attribute	Value
Recommended Model	Claude Sonnet 4.5
Reasoning	WebSocket/Redis pub-sub patterns are well-defined. Fast iteration needed for connection handling, notification persistence, and presence TTL.
Extended Thinking	Not required.

Files to Load

- apps/backend/src/notifications/notifications.gateway.ts
- apps/backend/src/notifications/notifications.service.ts
- apps/backend/src/notifications/notifications.controller.ts
- apps/backend/src/presence/presence.service.ts
- apps/backend/src/redis/redis.module.ts
- apps/frontend/src/hooks/useNotifications.ts
- apps/frontend/src/hooks/usePresence.ts
- apps/frontend/src/components/layout/NotificationBell.tsx
- apps/frontend/src/components/ui/OnlinePresenceDot.tsx
- apps/frontend/src/lib/socket.ts

Beast Mode Instruction

> "You are a senior distributed systems engineer auditing the SkillBarter real-time system. Key focus: TWO separate Redis clients (pub and sub must be separate ioredis instances — sharing one causes deadlock), notification persistence before WebSocket emit, and JWT validation in handleConnection. For every bug: assign severity and provide corrected code inline."

Testing Prompt — Phase 6 Test Scenarios

1. Redis Pub/Sub Architecture

- CRITICAL: Verify TWO separate ioredis instances exist (one for pub, one for sub)
- Verify: The same Redis client is NOT used for both publishing and subscribing
- Verify: Sub client calls client.subscribe() on initialization
- Verify: Pub client calls client.publish() to emit events
- RED FLAG: Single Redis client used for both pub and sub = deadlock risk — ioredis throws when you call subscribe() on a client that has published

2. WebSocket Event Catalog

TABLE 13 — WEBSOCKET EVENT CATALOG

Event Name	Emitted To	Trigger
session.booked	Mentor	Student books a session
session.started	Both participants	Both join LiveKit room
session.completed	Both participants	Both mark complete
session.completion_pending	Other participant	One party marks complete
session.disputed	Both participants	Dispute raised
bounty.answered	Poster	Answer submitted
bounty.accepted	Responder	Answer accepted
notification.new	Target user	Any notification created
user.online	Broadcast	User connects WebSocket
user.offline	Broadcast	User disconnects WebSocket

3. Notification Persistence

- CRITICAL: Verify notification is saved to DB BEFORE WebSocket emit
- Verify: If WebSocket emit fails, notification still exists in DB (DB is source of truth)
- Verify: Notification has fields: id, userId, type, message, isRead, createdAt
- Verify: GET /notifications returns unread notifications for authenticated user

- Verify: POST /notifications/read-all marks all as read

4. Online Presence TTL

- Verify: User connecting WebSocket sets presence in Redis: SET user:{userId}:online 1 EX 30
- Verify: Heartbeat mechanism refreshes TTL every ~25 seconds (before 30s expiry)
- Verify: User disconnecting removes presence key (or lets it expire naturally)
- Verify: GET /presence/:userId returns { online: true/false }
- Verify: OnlinePresenceDot shows green dot for online users, gray for offline

Critical Rules Checklist — Phase 6

69. TWO Redis clients (pub and sub are separate ioredis instances)
70. Heartbeat keeps online status alive (30s TTL refreshed every ~25s)
71. Notification saved to DB BEFORE WebSocket emit
72. JWT validated in handleConnection (unauthenticated connections rejected immediately)
73. Socket rooms use pattern user:{userId} not socket.id
74. Reconnection config present (reconnection: true, reconnectionAttempts set)
75. Unread badge count correct and updates in real-time
76. Mark-all-read clears badge count
77. Online presence dot shows/hides correctly
78. Fallback polling every 60s if WebSocket connection fails

Phase 7 — Admin Panel + Dispute Resolution

TABLE 14 — PHASE 7 MODEL CONFIGURATION

Attribute	Value
Recommended Model	Claude Opus 4.5 with Extended Thinking
Reasoning	Dispute resolution math (SPLIT) and ban transaction (cancelling all sessions + refunding escrows atomically) require deep reasoning.
Extended Thinking	REQUIRED — trace SPLIT math step by step.

Files to Load

- apps/backend/src/admin/admin.controller.ts
- apps/backend/src/admin/admin.service.ts
- apps/backend/src/disputes/disputes.service.ts

- apps/backend/src/disputes/disputes.controller.ts
- apps/backend/src/disputes/dto/resolve-dispute.dto.ts
- apps/backend/src/auth/guards/roles.guard.ts
- apps/backend/src/auth/decorators/roles.decorator.ts
- apps/frontend/src/pages/admin/AdminDashboardPage.tsx
- apps/frontend/src/components/admin/DisputeDetailModal.tsx
- apps/frontend/src/components/admin/UserManagementTable.tsx
- apps/frontend/src/components/admin/AdminStatsCards.tsx

Beast Mode Instruction

> "You are a senior security engineer auditing the SkillBarter admin panel. Enable extended thinking. For dispute resolution: 1. Trace the SPLIT math step by step (must sum to escrowAmount — no token creation or destruction). 2. Verify every admin route has BOTH @Roles('ADMIN') AND @UseGuards(JwtAuthGuard, RolesGuard). 3. Verify ban transaction atomicity (cancel sessions + refund escrows in ONE transaction). For every bug: assign severity and provide corrected code inline."

Testing Prompt — Phase 7 Test Scenarios

1. Admin Route Security (EVERY Route Must Be Checked)

- Verify: EVERY route in admin.controller.ts has @Roles('ADMIN')
- Verify: EVERY route in admin.controller.ts has @UseGuards(JwtAuthGuard, RolesGuard)
- Verify: EVERY route in disputes.controller.ts has @Roles('ADMIN') + @UseGuards(...)
- Attempt: Access GET /admin/users without JWT → must return 401
- Attempt: Access GET /admin/users with regular user JWT → must return 403
- Attempt: Access GET /admin/users with admin JWT → must return 200
- RED FLAG: Any admin route missing either guard = CRITICAL security vulnerability

2. Dispute Resolution — SPLIT Math Verification

TABLE 15 — SPLIT DISPUTE RESOLUTION MATH (TOKEN CONSERVATION MUST HOLD)

escrowAmount	tax (Math.floor × 0.05)	remaining	mentorReceives (floor/2)	studentReceives	Conservation Check
100	5	95	47	48	47+48+5 = 100 ✓
101	5	96	48	48	48+48+5 = 101 ✓
99	4	95	47	48	47+48+4 = 99 ✓

50	2	48	24	24	24+24+2 = 50 ✓
1	0	1	0	1	0+1+0 = 1 ✓

- Verify: SPLIT math is documented in code comments (half, tax, mentorReceives, studentReceives)
- Verify: All balance updates in single prisma.$transaction()
- Verify: resolvedAt set on escrow after SPLIT resolution
- Verify: Transaction records created for: mentor credit, student credit, tax to admin

3. User Ban — Atomic Transaction

- Simulate: Admin bans user X who has 2 pending sessions and 1 locked escrow
- Verify ALL of the following happen in SINGLE prisma.$transaction():
  - User.status → BANNED
  - All PENDING sessions where user is mentor → CANCELLED
  - All PENDING sessions where user is student → CANCELLED
  - All LOCKED escrows for cancelled sessions → REFUNDED to respective students
  - Transaction records created for each refund
- Verify: Cannot ban another ADMIN (role check returns 403)
- Verify: Banned user cannot log in (auth check on login)
- Verify: Banned user's active WebSocket connections are terminated

4. Redis SCAN vs KEYS

- CRITICAL: Search codebase for redis.keys('*') or client.keys('*')
- Verify: All Redis key enumeration uses SCAN with cursor (not KEYS)
- Reason: KEYS blocks the Redis event loop — in production with thousands of keys, this causes latency spikes and can freeze the entire Redis instance

Critical Rules Checklist — Phase 7

79. EVERY admin route has @Roles('ADMIN') + @UseGuards(JwtAuthGuard, RolesGuard)
80. SPLIT math: mentorReceives + studentReceives + tax = escrowAmount (documented in comments)
81. Ban cancels ALL pending sessions and refunds escrows in same transaction
82. adminNotes minimum 20 characters enforced (DTO + frontend)
83. Cannot ban another ADMIN (role check)
84. All admin actions auditable via Transaction table

85. Redis SCAN used (not KEYS) for daily active users
86. Admin dashboard charts render correctly
87. DisputeDetailModal confirm button disabled until notes >= 20 chars

---

Phase 8 — DevOps: Docker + Redis + CI/CD + Production Hardening

TABLE 16 — PHASE 8 MODEL CONFIGURATION

Attribute	Value
Recommended Model	Gemini 2.5 Pro
Reasoning	Phase 8 requires ingesting multiple large config files simultaneously. Gemini 2.5 Pro's 1M token context window handles this better than Claude.
Extended Thinking	Not required — use Gemini's large context instead.

Files to Load

- apps/backend/Dockerfile
- apps/frontend/Dockerfile
- docker-compose.yml
- docker-compose.prod.yml
- nginx/nginx.conf
- .github/workflows/ci.yml
- .github/workflows/deploy.yml
- apps/backend/src/main.ts (Pino logger, rate limiter, helmet config)
- apps/backend/src/config/configuration.ts (env var validation)
- apps/backend/.env.example
- apps/frontend/.env.example
- redis.conf (if exists)
- .gitignore

Beast Mode Instruction

> "You are a senior DevOps engineer auditing the SkillBarter production infrastructure. Load ALL files simultaneously — use your full context window. Check: multi-stage Dockerfiles, Nginx WebSocket config, CI/CD pipeline order, all 17 required env vars, Redis memory policy, Pino redaction, rate limiting. For every bug: assign severity and provide corrected code/config inline."

Testing Prompt — Phase 8 Test Scenarios

1. Environment Variables (All 17 Required)

TABLE 17 — REQUIRED ENVIRONMENT VARIABLES REFERENCE

#	Variable	Required	Notes
1	DATABASE_URL	Yes	PostgreSQL connection string
2	REDIS_URL	Yes	Redis connection string with password
3	JWT_ACCESS_SECRET	Yes	Min 32 chars — signs 15-minute access tokens
4	JWT_REFRESH_SECRET	Yes	Min 32 chars — signs 7-day refresh tokens
5	JWT_ACCESS_EXPIRY	Yes	Must be '15m'
6	JWT_REFRESH_EXPIRY	Yes	Must be '7d'
7	GITHUB_CLIENT_ID	Yes	GitHub OAuth App client ID
8	GITHUB_CLIENT_SECRET	Yes	GitHub OAuth App client secret
9	GITHUB_CALLBACK_URL	Yes	Must match GitHub OAuth App settings exactly
10	LINKEDIN_CLIENT_ID	Yes	LinkedIn OAuth App client ID
11	LINKEDIN_CLIENT_SECRET	Yes	LinkedIn OAuth App client secret
12	LINKEDIN_CALLBACK_URL	Yes	Must match LinkedIn OAuth App settings exactly
13	LIVEKIT_API_KEY	Yes	LiveKit project API key
14	LIVEKIT_API_SECRET	Yes	LiveKit project API secret
15	LIVEKIT_URL	Yes	LiveKit server URL (wss://...)
16	CLOUDINARY_URL	Yes (or S3 vars)	Cloudinary connection string
17	ADMIN_WALLET_USER_ID	Yes	Admin wallet DB user ID — never hardcode

2. Nginx WebSocket Configuration

- Verify: WebSocket proxy configured with proxy_http_version 1.1
- Verify: proxy_set_header Upgrade $http_upgrade present
- Verify: proxy_set_header Connection 'upgrade' present
- Verify: proxy_read_timeout 86400 (24 hours — CRITICAL for WebSocket)
- RED FLAG: Without proxy_read_timeout 86400, Nginx kills WebSocket connections after 60 seconds
- Verify: SSL/TLS configured (HTTPS in production)
- Verify: HTTP → HTTPS redirect configured
- Verify: Gzip compression enabled

3. GitHub Actions CI/CD Pipeline

- Verify: Pipeline order: lint → test → build → push → deploy
- Verify: Deploy job only runs after build job succeeds (needs: build-and-push)
- Verify: Health check passes before deploy job completes
- Verify: Docker image tagged with git SHA (not just 'latest')
- Verify: Secrets (JWT_SECRET, etc.) stored in GitHub Secrets (not in YAML)
- Verify: No sensitive values hardcoded in workflow YAML

4. Rate Limiting

- Verify: Auth endpoints rate limited to 5 requests per minute per IP
- Verify: Rate limiter applied to: POST /auth/login, POST /auth/register, POST /auth/refresh
- Verify: Rate limit exceeded returns 429 Too Many Requests
- Simulate: 6 rapid requests to POST /auth/login → 6th must return 429

Critical Rules Checklist — Phase 8

88. Multi-stage Dockerfiles (not single-stage — single-stage images are too large)
89. CMD runs 'prisma migrate deploy' before app start (never skip migrations)
90. .env not committed (.gitignore check — verify .env is listed)
91. Redis maxmemory-policy = allkeys-lru (prevents out-of-memory crashes)
92. Pino redacts authorization headers and cookies (never log tokens)
93. Auth endpoints rate limited to 5/min (brute force protection)
94. Health check passes before deploy job runs
95. Nginx WebSocket proxy_read_timeout = 86400
96. All 17 required env vars present and validated on startup
97. GitHub Actions pipeline order: lint → test → build → push → deploy

98. docker-compose has all 5 services: postgres, redis, backend, frontend, nginx

Section 3 — The Mega End-to-End Prompt

TABLE 18 — MEGA E2E PROMPT CONFIGURATION

Attribute	Value
Primary Model	Gemini 2.5 Pro
Alternative Model	Claude Opus 4.5 with Extended Thinking
Files to Load	ENTIRE CODEBASE — all files from all 8 phases simultaneously
Extended Thinking	REQUIRED for Claude Opus 4.5 alternative

Beast Mode Instruction

> "You are a senior full-stack engineer performing a complete production readiness audit of SkillBarter. Load the entire codebase. Simulate the complete user journey from registration to dispute resolution. For every bug found: 1. Assign severity: CRITICAL / HIGH / MEDIUM / LOW. 2. Fix it inline with corrected code. 3. Verify token conservation (no tokens created or destroyed outside of defined rules). Enable extended thinking. This is a comprehensive audit — do not skip any step."

Part 1 — The Golden Path Simulation (15 Steps)

Execute each step in sequence. For each step, verify the expected outcome. Report any deviation as a bug with severity and inline fix.

TABLE 19 — GOLDEN PATH SIMULATION: 15-STEP SEQUENCE

Step	Action	Key Verifications	BUG CHECK
1	POST /auth/register { email: alice@test.com, password, name }	Response has user object (no passwordHash), access token in body, httpOnly refresh cookie, DB: token_balance=50, Transaction EARN record created	localStorage.getItem('token') must return null
2	Alice connects GitHub OAuth (first time)	+50 coins credited (balance=100), Transaction EARN record created	alice.token_balance must be exactly 100
3	Alice connects GitHub OAuth (second time)	0 additional coins (idempotent)	alice.token_balance must still be exactly 100
4	Alice connects LinkedIn OAuth (first time)	+50 coins credited (balance=150)	alice.token_balance must be exactly 150

5	Bob registers + lists 'Advanced TypeScript' skill at 30 coins	Skill appears in FTS search for 'typescript', GIN index used	Search uses plainto_tsquery not LIKE
6	Alice books Bob's session (30 coins, 1 hour from now)	ATOMIC: escrow locked + session created in single transaction, Alice balance=120, Bob notified via WebSocket	If escrow lock succeeds but session.create fails, Alice's 30 coins must be rolled back
7	Both join classroom 10 min before start	LiveKit tokens generated, session status → ACTIVE when both join, countdown timer synced	Token generation is time-gated on backend
8	Bob clicks 'Mark Complete'	mentor_completed=true, escrow still LOCKED, student notified	Session status must NOT change to COMPLETED yet
9	Alice clicks 'Mark Complete'	ATOMIC: session→COMPLETED, escrow→RELEASED, tax=1 to admin, Bob+=29, Bob.teachingStreak++, resolvedAt set	Total tokens unchanged: 30 = 29 (Bob) + 1 (admin)
10	Alice submits 5-star rating	bob.reputation_score updated (rolling average), cannot rate same session twice	Rating modal appeared after completion, not before
11	Alice creates bounty 'Redis pub/sub?' reward=20 coins	ATOMIC: escrow locked + bounty created, Alice balance=100	Alice balance must be exactly 100
12	Bob uploads video answer via presigned URL	File goes DIRECTLY to S3/Cloudinary, Bob sends only videoUrl to NestJS	No @UploadedFile() on answer endpoint
13	Alice accepts Bob's answer	ATOMIC: escrow→RELEASED, tax=1 to admin, Bob+=19, bounty→CLOSED	Bob balance: 50+29+19=98
14	Carol books Dave (50 coins), Carol raises dispute	Dispute created, escrow→DISPUTED, resolvedAt NOT set, admin notified	Session cannot be completed while disputed
15	Admin resolves dispute with SPLIT, adminNotes >= 20 chars	ATOMIC: tax=2, Carol+=24, Dave+=24, escrow→RELEASED, resolvedAt set	24+24+2=50 — token conservation verified

Part 2 — Security Audit

2.1 Authentication Bypass Attempts

- Attempt: GET /sessions without JWT → must return 401
- Attempt: GET /admin/users without JWT → must return 401
- Attempt: GET /admin/users with regular user JWT → must return 403
- Attempt: POST /sessions with manipulated JWT (changed userId) → must return 401

2.2 Data Exposure

- Attempt: GET /users/:id → verify passwordHash NOT in response
- Attempt: GET /skills → verify no user.passwordHash in nested user objects
- Attempt: GET /sessions → verify no passwordHash in mentor/student objects

2.3 Token Overdraft & SQL Injection

- Attempt: Book session costing 200 coins with balance of 50 → must return 400 InsufficientFunds
- Attempt: Create bounty for 200 coins with balance of 50 → must return 400 InsufficientFunds
- Verify: token_balance CHECK constraint prevents negative values at DB level
- Attempt: Search with payload: "'; DROP TABLE skills; --" → sanitized, no error
- Attempt: Search with payload: "' OR '1'='1" → sanitized, no error
- Verify: Prisma parameterized queries prevent injection (no raw SQL with string interpolation)

Part 3 — Token Conservation Audit

> Token Conservation Law: sum(all user balances) + sum(all LOCKED escrows) + sum(all TAX transactions) = sum(all EARN transactions). Any discrepancy indicates a bug in LedgerService.

TABLE 20 — EXPECTED TOKEN STATE AFTER GOLDEN PATH SIMULATION

User	Starting Balance	Transactions	Expected Final Balance
Alice	50 (signup) + 50 (GitHub) + 50 (LinkedIn) = 150	−30 (session booking) − 20 (bounty creation)	100 coins
Bob	50 (signup)	+29 (session completion) + 19 (bounty acceptance)	98 coins
Admin Wallet	0	+1 (session tax) + 1 (bounty tax) + 2 (dispute tax)	4 coins

Carol	50 (signup)	−50 (session booking) + 24 (dispute split)	24 coins
Dave	50 (signup)	+24 (dispute split)	74 coins

Part 4 — Production Readiness Check

- Verify: GET /health returns 200 { status: 'ok', db: 'connected', redis: 'connected' }
- Verify: All 17 env vars present and validated (app throws on startup if missing)
- Verify: Pino NOT logging JWT tokens or passwords
- Verify: 6th auth request in 1 minute returns 429 Too Many Requests
- Verify: Nginx WebSocket proxy_read_timeout = 86400
- Verify: No TypeScript 'any' types in financial code (LedgerService, WalletService)
- Verify: All Prisma transactions atomic (no sequential awaits in financial operations)

Deliverable Format

The AI model should produce a structured report with the following sections:

99. Executive Summary: Total bugs found (CRITICAL/HIGH/MEDIUM/LOW counts), Token Conservation PASS/FAIL, Security Audit PASS/FAIL, Production Readiness PASS/FAIL
100. Bug Report Table sorted by severity: # | Severity | Phase | File | Description | Fix
101. Corrected Code for every CRITICAL and HIGH bug (inline, not described)
102. Token Conservation Proof: total_issued = current_balances + locked_escrows + burned_tax
103. Race Condition Analysis: concurrent booking, concurrent escrow release
104. TypeScript Type Safety Report: any 'any' types found in financial code

Section 4 — UX Quality Gates Checklist

Review all frontend pages and components against the following quality gates. For each item: verify PASS or FAIL. For every FAIL: provide the corrected component code inline.

4.1 Loading States

TABLE 21 — LOADING STATE QUALITY GATES

Component	Expected Loading State	Pass/Fail
MarketplacePage	Shadcn Skeleton cards while fetching skills	[ ]
WalletPage — BalanceCard	Skeleton placeholder matching card dimensions	[ ]
SessionsListPage	Skeleton rows in session list	[ ]
BountyBoardPage	Skeleton bounty cards	[ ]
AdminDashboardPage	Skeleton stats cards + chart placeholders	[ ]
All async operations	No blank screens — always show Skeleton	[ ]

4.2 Error Boundaries

- Error boundary wraps all page-level components
- Error boundary shows user-friendly message (not raw error or stack trace)
- Error boundary has 'Try Again' button that resets the boundary
- Error boundary does NOT expose stack traces in production (NODE_ENV check)

4.3 Toast Notifications

TABLE 22 — TOAST NOTIFICATION QUALITY GATES

Action	Expected Toast	Variant
Successful booking	'Session booked successfully!'	success
Failed booking (insufficient balance)	'Insufficient balance. You need X more coins.'	error
Successful bounty creation	'Bounty posted! Reward locked in escrow.'	success
Dispute raised	'Dispute raised. Admin has been	info

	notified.'	
Session completed	'Session completed! Tokens released to mentor.'	success
Any API error	Specific error message from API response	error
Silent failure (no toast)	FAIL — all errors must surface to user	CRITICAL

4.4 Mobile Responsiveness (375px Width)

- Marketplace page: skill cards stack vertically (1 column on mobile)
- Wallet page: summary cards stack vertically
- Classroom page: video only on mobile (editor/whiteboard hidden or tabbed)
- Admin panel: tables scroll horizontally (not clipped)
- Navigation menu: collapses to hamburger on mobile

4.5 Empty States

TABLE 23 — EMPTY STATE QUALITY GATES

Page/Component	Expected Empty State Message	CTA Button?
MarketplacePage (no skills)	'No skills found. Be the first to list a skill!'	Yes — 'List a Skill'
SessionsListPage (no sessions)	'No sessions yet. Browse the marketplace to book your first session!'	Yes — 'Browse Marketplace'
NotificationsPage (no notifications)	'You're all caught up! No new notifications.'	No
BountyBoardPage (no bounties)	'No bounties posted yet. Post a question to get started!'	Yes — 'Post Bounty'
WalletPage (no transactions)	'No transactions yet. Your token history will appear here.'	No

4.6 Disabled States

- BookingModal 'Book Now' disabled when balance < price (with tooltip: 'You need X more coins')

- DisputeDetailModal confirm button disabled until adminNotes.length >= 20 (with character count shown)
- 'Join Room' button disabled > 10 min before session start (with countdown to unlock)
- All submit buttons disabled during API calls (prevent double-submit)
- Rating submit disabled until star rating selected (1–5)

4.7 Accessibility

- All interactive elements have aria-label or aria-labelledby
- Focus management: modal opens → focus trapped inside → modal closes → focus returns to trigger element
- Keyboard navigation works for all modals (Escape to close, Tab to navigate)
- Color contrast meets WCAG AA (4.5:1 for normal text, 3:1 for large text)
- Images have descriptive alt text (not empty alt='')

4.8 Visual Quality

TABLE 24 — VISUAL QUALITY GATES

Element	Expected Behavior	Pass/Fail
WalletPage BalanceCard	Animated count-up on mount (framer-motion)	[ ]
Transaction badges	EARN=green, SPEND=red, ESCROW_LOCK=amber, ESCROW_RELEASE=green, TAX=gray, REFUND=blue	[ ]
Online presence dot	Green = online, gray = offline (no yellow or other colors)	[ ]
Notification badge	Red dot with unread count, hidden when count = 0	[ ]
Skeleton dimensions	Skeleton matches final content dimensions (no layout shift)	[ ]
Optimistic updates	Mark notification read → immediate UI update (not waiting for API)	[ ]

Section 5 — Token Conservation Audit SQL Queries

Run these queries directly against the PostgreSQL database to verify token conservation, detect anomalies, and audit the financial integrity of the SkillBarter platform. All queries should be run in a read-only transaction to prevent accidental modifications.

> Run these queries after every major deployment and after any dispute resolution. A non-zero discrepancy in Query 2 indicates a critical bug in LedgerService that must be investigated immediately.

Query 1 — Current State Summary

Provides a high-level snapshot of all token flows: user balances, locked escrows, tax burned, and bonuses issued. Use this as the first diagnostic query.

TABLE 25 — QUERY 1: CURRENT STATE SUMMARY METRICS

Metric	SQL Clause	Expected Behavior
Total User Balances	SUM(token_balance) WHERE status != 'BANNED'	Sum of all active user balances
Locked Escrow Amounts	COALESCE(SUM(amount), 0) WHERE status = 'LOCKED'	Tokens currently in escrow
Total Tax Burned (to Admin)	COALESCE(SUM(amount), 0) WHERE type = 'TAX'	Cumulative tax collected
Total Signup Bonuses Issued	SUM(amount) WHERE type = 'EARN' AND description LIKE '%signup%'	Total signup bonuses
Total OAuth Bonuses Issued	SUM(amount) WHERE type = 'EARN' AND description LIKE '%OAuth%'	Total OAuth bonuses

Query 2 — Token Conservation Formula

The master conservation check. The discrepancy column MUST be 0. Any non-zero value indicates tokens were created or destroyed outside of defined rules — a critical financial bug.

> Conservation Law: total_issued = current_balances + locked_escrows + burned_tax. If discrepancy != 0, check LedgerService for non-atomic operations (sequential awaits instead of prisma.$transaction()).

Query 3 — Detect Negative Balances

Should return 0 rows in a healthy system. Any row returned indicates the application-level balance check failed AND the DB CHECK constraint was bypassed — a critical double failure.

TABLE 26 — QUERY 3: NEGATIVE BALANCE DETECTION

Query	Expected Result	If Non-Zero
SELECT id, email, token_balance FROM users WHERE token_balance < 0	0 rows	CRITICAL: Application-level check AND DB constraint both failed
SELECT id, email, token_balance FROM users WHERE token_balance IS NULL	0 rows	HIGH: token_balance has no default or was set to NULL
SELECT COUNT(*) FROM escrows WHERE status = 'LOCKED' AND amount <= 0	0 rows	HIGH: Escrow created with invalid amount

Query 4 — Orphaned Escrows

Detects LOCKED escrows that have no associated session or bounty. These represent tokens that are permanently locked with no way to release them — a critical data integrity issue.

Query 5 — Escrow State Machine Violations

Detects escrows that violate the one-way state machine: LOCKED escrows with resolvedAt set (should never happen), or RELEASED/REFUNDED escrows without resolvedAt (should always be set).

TABLE 27 — QUERY 5: ESCROW STATE MACHINE VIOLATION TYPES

Violation Type	Detection Condition	Root Cause
LOCKED with resolvedAt set	status = 'LOCKED' AND resolved_at IS NOT NULL	freezeEscrow incorrectly set resolvedAt
RELEASED without resolvedAt	status = 'RELEASED' AND resolved_at IS NULL	releaseEscrow did not set resolvedAt
REFUNDED without resolvedAt	status = 'REFUNDED' AND resolved_at IS NULL	refundEscrow did not set resolvedAt
DISPUTED with resolvedAt set	status = 'DISPUTED' AND resolved_at IS NOT NULL	freezeEscrow incorrectly set resolvedAt

Query 6 — Duplicate OAuth Bonuses (Idempotency Check)

Detects users who received the same OAuth bonus more than once — indicating the idempotency check in the OAuth strategy failed. Should return 0 rows.

Query 7 — Admin Wallet Integrity

Verifies the admin wallet is receiving all tax revenue correctly. The TAX transaction count should match the total number of completed sessions and accepted bounties.

Query 8 — Daily Token Velocity (Anomaly Detection)

Tracks token flow by day and transaction type over the last 30 days. Use this to detect anomalies: sudden spikes in EARN transactions (bonus exploit), unusual ESCROW_LOCK patterns (booking fraud), or TAX amounts inconsistent with session volume.

TABLE 28 — QUERY 8: TOKEN VELOCITY ANOMALY PATTERNS

Anomaly Pattern	What It Indicates	Severity
EARN transactions >> expected signup/OAuth volume	Bonus exploit — idempotency check bypassed	CRITICAL
ESCROW_LOCK without matching ESCROW_RELEASE or REFUND	Orphaned escrows — tokens permanently locked	HIGH
TAX amount inconsistent with session count	Tax calculation bug in LedgerService	HIGH
PENALTY transactions with amount > 10	No-show penalty cap not enforced	MEDIUM
BOUNTY transactions without matching ESCROW_LOCK	Bounty reward paid without escrow	CRITICAL

SkillBarter Beast Mode Testing Prompt Suite — Department of Computer Engineering, University of Mumbai, Academic Year 2025–2026. Submitted by Adit Halde (337), Kartik Hulmukh (338), Vivek Humbe (339). Supervised by Prof. Kaajal Sharma.